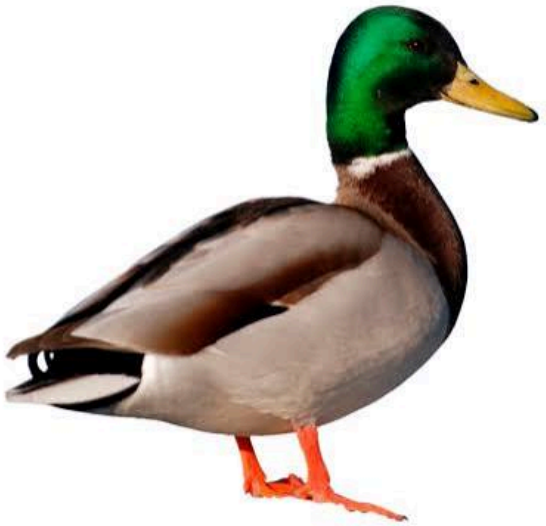
Ciência de Dados Aplicada ao Direito I

Aula 19

Bibliotecas e modelos de linguagem



Prof. Bruno da Cunha de Oliveira - 2026/2

O que tem num repositório

Insper

README: o manual do projeto

O `README.md` é a primeira coisa que aparece na página de um repositório. Diz o que o projeto faz, como instalar e como rodar.

O que o grupo não entendeu, a IA ajuda a entender: colem o trecho num chat e peçam a explicação para alguém que nunca programou. Depois confiram no repositório se a explicação bate com o que está lá.

Na fase 2, o `README` é a porta de entrada do repositório que vocês entregam à banca.

Licença: o que se pode fazer com o código

Licença	O que permite, e o que pede em troca
MIT	usar, mudar, vender; manter o aviso e a licença
Apache 2.0	o mesmo, e dizer o que foi mudado
BSD 3-Clause	o mesmo, sem usar o nome dos autores para promover
GPL e AGPL	quem distribui versão mudada abre o código; na AGPL, também quem a oferece pela internet
proprietária	só quem o dono autorizar usa, mesmo com o código à vista
nenhuma	o mesmo que a proprietária

A licença do grupo: três caminhos

Caminho	Como fica
aberto	seguir com a licença da base, ou escolher uma no GitHub: **Add file**, **Create new file**, o nome `LICENSE` e o botão **Choose a license template**
fechado	licença proprietária: o repositório continua público para a banca, e só quem o grupo autorizar usa; o `LICENSE` diz "Copyright 2026 Grupo X. Todos os direitos reservados."
misto	uma parte aberta e outra fechada

Com uma base MIT, o grupo pode fechar o produto

O CÓDIGO DO GRUPO

a licença que o grupo escolher, inclusive a proprietária

O CÓDIGO QUE VEIO DA BASE

continua MIT, e o aviso de copyright da base fica no repositório

MIT, Apache 2.0 e BSD pedem só que o aviso e a licença da base fiquem junto do código que veio dela.

Com uma base GPL ou AGPL seria diferente: o produto inteiro teria de ficar aberto. Nenhuma das cinco bases é assim.

Misto: o PrairieLearn

O PrairieLearn, em que vocês fazem os notebooks, tem o código inteiro no GitHub, com três licenças:

Parte	Licença	O que permite
edição comunitária	AGPL	usar e mudar; quem oferece pela internet abre o código
edição Enterprise	proprietária	usar em produção só com contrato ou assinatura
código antigo, de Illinois	MIT	usar, mudar e vender, mantendo o aviso

Issue e pull request

Aba	O que tem nela
Issues	conversas sobre o projeto: uma dúvida, uma sugestão, um defeito encontrado. Não muda arquivo nenhum
Pull requests	propostas de mudança nos arquivos, que o dono lê, comenta e aceita ou recusa. A sigla é PR

AGENTS.md e CLAUDE.md: instruções para o assistente

Arquivo	O que é
`AGENTS.md`	um formato aberto de instruções para assistentes de código; o Codex e outros leem
`CLAUDE.md`	as instruções que o Claude Code lê no começo de cada sessão; sem ele, o Claude Code lê o `AGENTS.md`

Na APS: escrevam ali o que o produto faz, como rodar e as regras do grupo, como "responda em português". O assistente de cada um de vocês começa sabendo, sem que ninguém precise repetir.

Registrar e enviar

Insper

Conferir o Git e a conta no GitHub

O que conferir	Está certo quando
No terminal do VS Code, `git --version`	responde `git version` e um número
No navegador, `github.com`	abre com a sua conta, e o seu ícone no alto à direita

Quem ainda não fez: passos 1 e 2 do guia, no Blackboard. No Windows, baixar o instalador do Git e reabrir o VS Code; no Mac, `xcode-select --install`.

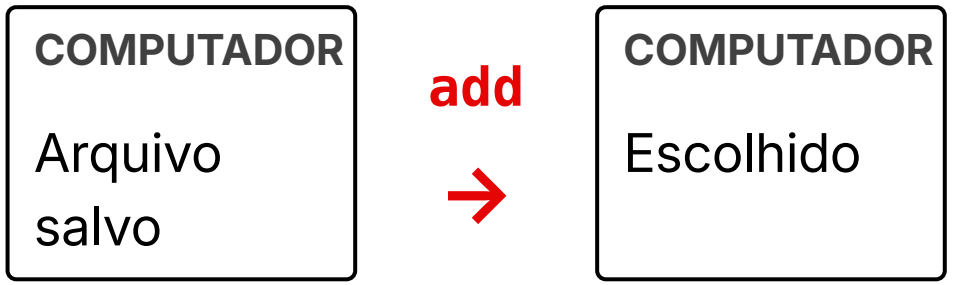
Dizer ao Git quem você é

No terminal do VS Code, uma linha de cada vez, com o seu nome e o seu e-mail entre aspas:

```
git config --global user.name "Seu Nome"
git config --global user.email "voce@exemplo.com"
```

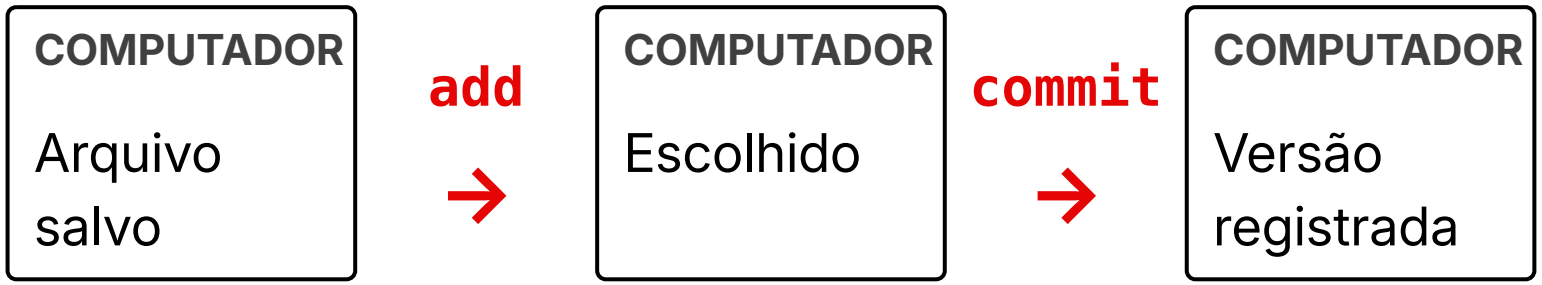
As duas linhas não respondem nada. Sem elas, o `git commit` para com "Please tell me who you are". Toda versão registrada leva esse nome e esse e-mail.

add, commit e push



`git add .` escolhe tudo o que mudou na pasta para entrar na próxima versão.

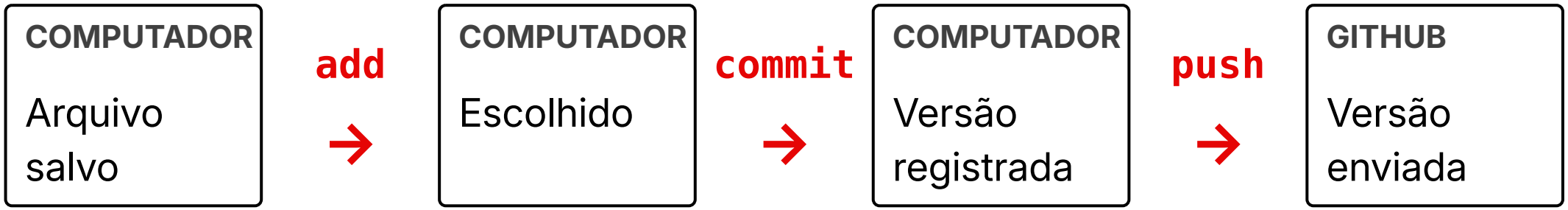
add, commit e push



`git add .` escolhe tudo o que mudou na pasta para entrar na próxima versão.

`git commit -m "..."` registra a versão no computador, com uma mensagem que diz o que mudou.

add, commit e push



`git add .` escolhe tudo o que mudou na pasta para entrar na próxima versão.

`git commit -m "..."` registra a versão no computador, com uma mensagem que diz o que mudou.

`git push` envia ao GitHub as versões registradas. Sem ele, ninguém do grupo nem da banca as vê.

Duas atividades no GitHub

Insper

Atividade 1: uma issue e um pull request

#	No repositório `bdcdo/praticas-github-cdad-2026-2`
1	Abrir uma issue explicando o que é uma issue, como você explicaria a um advogado que nunca programou
2	Fork, clone e, no VS Code, o arquivo `explicacoes/seu-usuario.md`, explicando o que é um pull request ao mesmo advogado
3	Registrar e enviar com `git add`, `git commit` e `git push`, e abrir o pull request a partir do fork

Até o início da aula de terça, 27/10, pelo formulário "issue e PR", no Blackboard. Escritos por você, sem assistente; o passo a passo está no guia.

Quanto vale a atividade 1

Quem	Ponto extra
Abriu a issue e o pull request	0,1
A melhor explicação de issue	mais 0,1
A melhor explicação de pull request	mais 0,1

O ponto extra vai para a prova final. A mesma pessoa pode ganhar os dois prêmios e chegar a 0,3. No empate, ganha quem enviou primeiro.

Atividade 2: o seu site pessoal

#	O site tem
1	Endereço `seu-usuario.github.io`, abrindo para qualquer pessoa
2	Uma apresentação sua
3	Link para o seu currículo Lattes
4	Link para o seu perfil no LinkedIn

Quanto vale a atividade 2

Link	Está atualizado quando
Lattes	a "Última atualização" é de 2026
LinkedIn	o perfil tem foto, título e a formação no Insper

Vale 0,1 ponto extra na prova final para quem tiver os quatro itens no site e os dois links atualizados até o início da aula de quarta, 25/11, e mandar o endereço por e-mail a `brunoco2@insper.edu.br`. O passo a passo está no guia.

O assistente de código

Insper



Andrej Karpathy
@karpathy

There's a new kind of coding I call "vibe coding", where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. It's possible because the LLMs (e.g. Cursor Composer w Sonnet) are getting too good. Also I just talk to Composer with SuperWhisper so I barely even touch the keyboard. I ask for the dumbest things like "decrease the padding on the sidebar by half" because I'm too lazy to find it. I "Accept All" always, I don't read the diffs anymore. When I get error messages I just copy paste them in with no comment, usually that fixes it. The code grows beyond my usual comprehension, I'd have to really read through it for a while. Sometimes the LLMs can't fix a bug so I just work around it or ask for random changes until it goes away. It's not too bad for throwaway weekend projects, but still quite amusing. I'm building a project or webapp, but it's not really coding - I just see stuff, say stuff, run stuff, and copy paste stuff, and it mostly works.

8:17 PM · Feb 2, 2025 · **4.9M** Views



Andrej Karpathy
@karpathy

Existe um novo tipo de codificação que eu chamo de “vibe coding”, onde você se entrega totalmente às vibrações, abraça os exponenciais e esquece que o código sequer existe. É possível porque os LLMs (por exemplo, Cursor Composer com Sonnet) estão ficando bons demais. Também falo com o Composer com SuperWhisper, então mal toco no teclado. Peço as coisas mais bobas como “reduza o preenchimento da barra lateral pela metade” porque sou preguiçoso demais para encontrar. Sempre clico em “Aceitar Tudo”, não leio mais as diferenças. Quando recebo mensagens de erro, apenas copio e colo nelas, sem comentário, geralmente isso resolve. O código cresce além da minha compreensão habitual, eu teria que realmente ler com atenção por um tempo. Às vezes os LLMs não conseguem corrigir um bug, então eu só contorno ou peço mudanças aleatórias até funcionar. Não é tão ruim para projetos de fim de semana que vou descartar depois, mas ainda assim é bem divertido. Estou construindo um projeto ou aplicativo web, mas não é realmente codificação – eu apenas vejo as coisas, digo coisas, executo coisas e copio e colo coisas, e na maioria das vezes funciona.

20:17 · 2 de fev de 2025 · **4,9 mi** Visualizações

The one thing I'd add is that at the time, LLM capability was low enough that you'd mostly use vibe coding for fun throwaway projects, demos and explorations. It was good fun and it almost worked. Today (1 year later), programming via LLM agents is increasingly becoming a default workflow for professionals, except with more oversight and scrutiny. The goal is to claim the leverage from the use of agents but without any compromise on the quality of the software. Many people have tried to come up with a better name for this to differentiate it from vibe coding, personally my current favorite "agentic engineering":

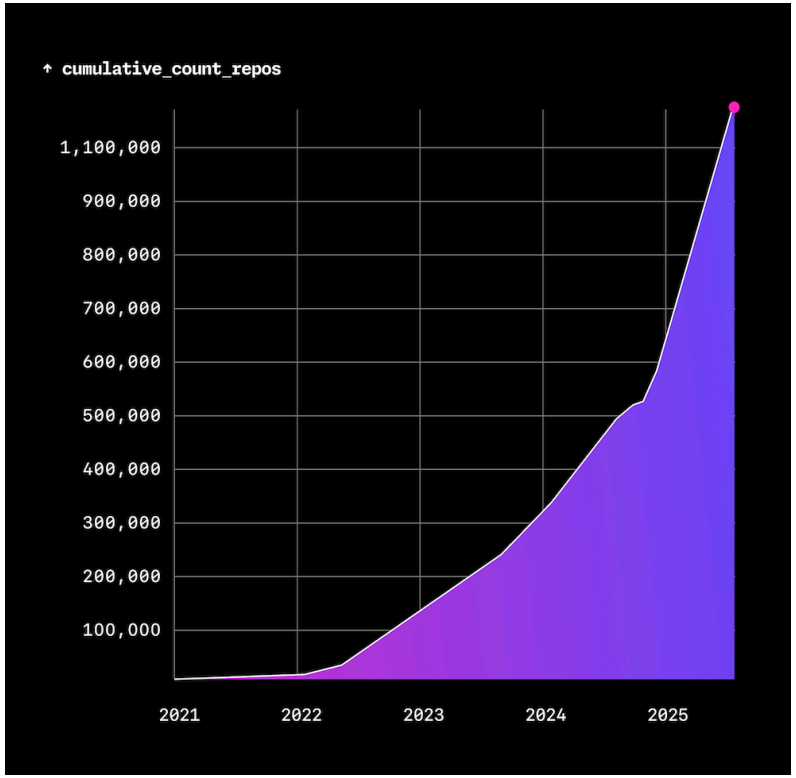
- "agentic" because the new default is that you are not writing the code directly 99% of the time, you are orchestrating agents who do and acting as oversight.
- "engineering" to emphasize that there is an art & science and expertise to it. It's something you can learn and become better at, with its own depth of a different kind.

In 2026, we're likely to see continued improvements on both the model layer and the new agent layer. I feel excited about the product of the two and another year of progress.

A única coisa que eu acrescentaria é que, na época, a capacidade dos LLMs era baixa o suficiente para que você usasse vibe coding principalmente em projetos divertidos e descartáveis, demos e explorações. Era divertido e quase funcionava. Hoje (1 ano depois), programar por meio de agentes de LLM está se tornando cada vez mais um fluxo de trabalho padrão para profissionais, só que com mais supervisão e escrutínio. O objetivo é obter a alavancagem do uso de agentes sem comprometer a qualidade do software. Muita gente tentou encontrar um nome melhor para isso, para diferenciar de vibe coding; pessoalmente, meu favorito no momento é “engenharia agêntica”:

- “agêntica” porque o novo padrão é que você não está escrevendo o código diretamente 99% do tempo; você está orquestrando agentes que fazem isso e atuando como supervisão.
- “engenharia” para enfatizar que há arte, ciência e expertise nisso. É algo que você pode aprender e em que pode se tornar

Nunca se criou tanto repositório



Projetos públicos no GitHub que usam modelo de IA, acumulado. GitHub, Octoverse 2025.

Insper

Antes do assistente: o clone do seu fork

O assistente trabalha numa pasta do computador. Quem ainda não tem a pasta `pratica-git`: fork em `github.com/bdcdo/pratica-git` e, no terminal do VS Code,

```
cd ~
git clone https://github.com/SEU-USUARIO/pratica-git.git
```

Deu certo quando a resposta começa com `Cloning into 'pratica-git'...`, sem a palavra `fatal`.

Passo 6 do guia: Codex ou Claude

Caminho	Onde roda	Que conta pede
Codex, aplicativo	programa separado	ChatGPT, até a gratuita
Codex, extensão	dentro do VS Code	ChatGPT Edu, Plus ou maior
Claude, aplicativo	à parte, aba **Code**	Claude Pro ou Max
Claude Code, extensão	dentro do VS Code	Claude Pro ou Max

Com o ChatGPT Edu do Insper, o Codex funciona no aplicativo e na extensão. Escolham um, instalem e abram nele a pasta `pratica-git`.

Se o assistente não registrar ou não enviar

Se acontecer	O que fazer
o assistente pede permissão para registrar ou enviar	aprovar
o envio feito pelo assistente falha	rodar `git push` no terminal do VS Code

Um pedido ao assistente

Façam este pedido ao assistente:

> Acrescente ao `prazos.py` a função `dias_restantes(prazo, decorridos)`, que devolve quantos dias faltam. Depois mostre quantos dias restam de um prazo de 15 dias em dobro, passados 12 dias. Registre a versão e envie ao GitHub.

Antes de rodar o programa: que número novo tem que aparecer?

A resposta antes do código

RESPOSTA

```
30
60
18
```

As duas primeiras linhas já estavam no arquivo. A nova é 18: o dobro de 15 é 30, e 30 menos 12 é 18.

Vocês sabem a resposta antes de o assistente escrever uma linha, e é isso que permite conferir o que ele entregou.

Pedir ao assistente que desfaça

A função nova não serve mais. Façam este pedido:

> Desfaça o último commit com um commit novo, sem apagar o histórico, e envie ao GitHub.

Quantas versões o histórico do seu fork passa a ter? A função `dias_restantes` some do histórico?

Desfazer acrescenta uma versão

RESPOSTA

Cinco: as duas que vieram do repositório do curso, a de vocês, a do assistente e a que desfaz.

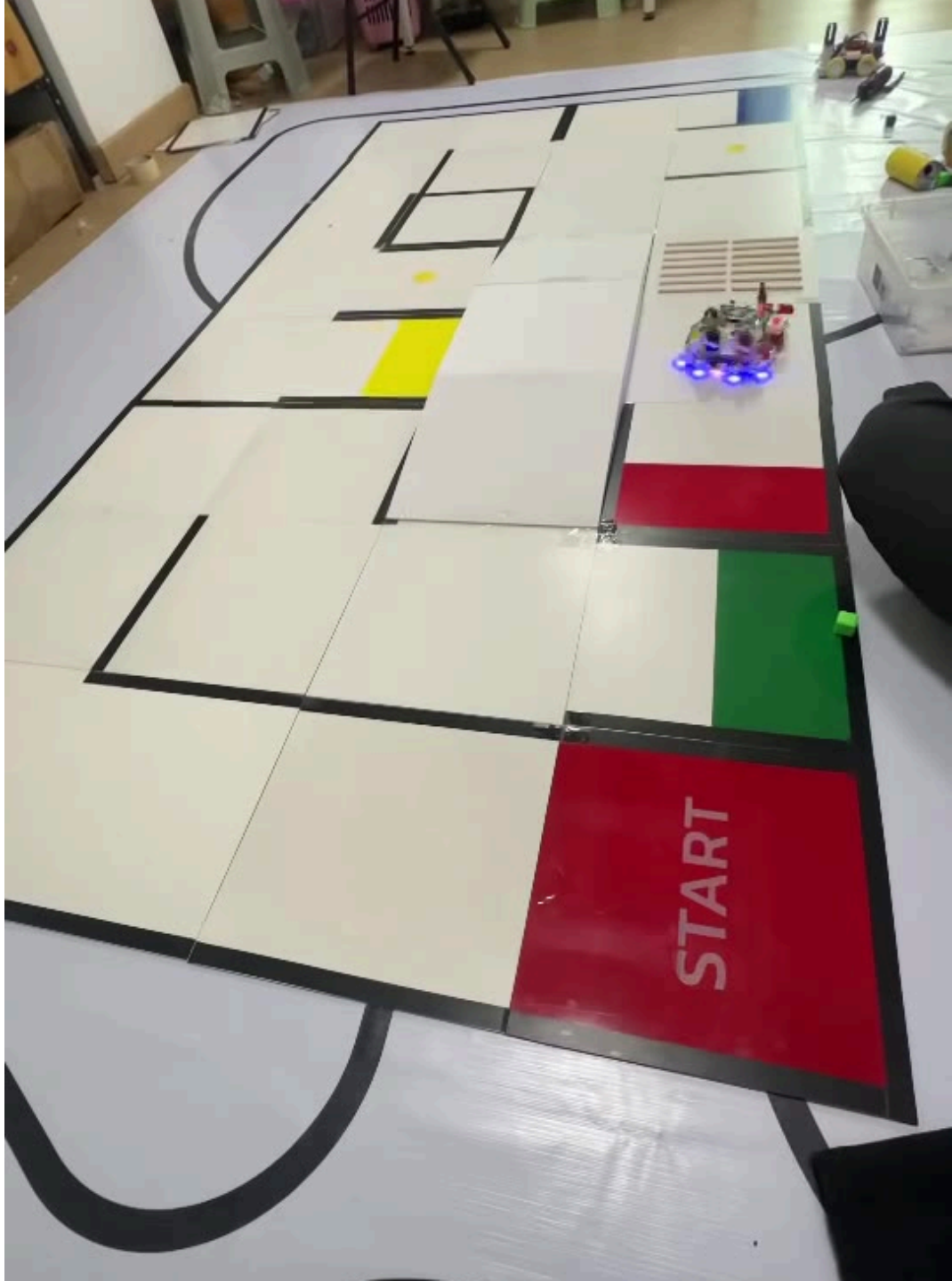
A função continua no histórico, na quarta versão. Desfazer não apaga nada: registra uma versão nova com o conteúdo de antes.

> Entreguem o endereço do seu fork pelo formulário "fork da aula 18", no Blackboard.

Bibliotecas

Robô autônomo num labirinto

Repetir e decidir vocês já escrevem em Python. O que ainda falta para este programa existir?



Robô autônomo num labirinto

Animação no navegador: `animacao-video-robo.html`.

Biblioteca: código que outra pessoa escreveu

No programa do juscraper	O que fez
`uv add juscraper`	instalou, uma vez
`import juscraper as jus`	deixou o programa usar
`tjsp = jus.scraper("tjsp")`	criou o raspador do TJSP
`tjsp.cjpg("golpe do pix")`	devolveu a tabela

A parte que fala com o tribunal ninguém aqui escreveu: ela está dentro da biblioteca.

Por que biblioteca importa com IA

Uma IA que escreve Python usa as bibliotecas que já existem: para consultar um tribunal, ler uma planilha, abrir um PDF, desenhar um gráfico, conversar com um modelo de linguagem.

Quase tudo já tem biblioteca. Por isso, com um bom pedido, a IA faz quase qualquer coisa por vocês: vocês dizem o que fazer, a biblioteca sabe fazer.

De onde veio o juscraper

O `uv add juscraper` baixou o pacote do PyPI, o catálogo público de bibliotecas de Python: `pypi.org/project/juscraper`. A página diz a versão, 0.4.0, e de que outras bibliotecas ela precisa:

```
pandas<3.0.0,>=2.0.0
requests>=2.32.3
```

Uma biblioteca usa outras. O uv instalou o pandas e o requests junto com o juscraper, nas versões que ele aceita.

juscraper 0.4.0

Raspador de tribunais e outros sistemas relacionados ao poder judiciário.

```
pip install juscraper
```


juscraper



Raspador de tribunais e outros sistemas relacionados ao poder judiciário brasileiro.

📦 Instalação

Via PyPI (Recomendado)

```
pip install juscraper
```

Com uv

Links do projeto

Data verified by PyPI on Sep 15, 2026

Bug Tracker
Changelog
Documentation
Homepage
Repository

Key dates

PyPI data

Released: 15 de set. de 2026

VERSÃO MAIS RECENTE

Nem toda biblioteca está no PyPI

O raspe coleta de fontes oficiais e da imprensa: Presidência, Câmara, Senado, IPEA, CAPES e Folha, entre outras. Ele não está no PyPI, e se instala direto do GitHub:

```
uv add git+https://github.com/bdcdo/raspe.git
```

```
import raspe
camara = raspe.camara()
dados = camara.raspar(pesquisa="educação")
```

Uma classe em cima da outra

No juscraper 0.4.0	Acrescenta
`class BaseScraper:`	o que todo raspador tem
`class HTTPScraper(BaseScraper):`	pedir páginas
`class EsajSearchScraper(HTTPScraper):`	o `cjsg` do e-SAJ
`class TJACScraper(EsajSearchScraper):`	o endereço do Acre

Entre parênteses, a classe de cima. Cada classe herda o que a de cima tem: `tjac.cjsg(...)` funciona sem uma linha de `cjsg` no raspador do Acre. O `cjsg` é a busca de segundo grau.

Escrever uma vez, usar em seis tribunais

Herdam de `EsajSearchScraper`	O que cada um escreve
`TJACScraper`, `TJALScraper`, `TJAMScraper`, `TJMSScraper`	só o endereço
`TJCEScraper`	o endereço e um ajuste de conexão
`TJSPScraper`	o endereço, um `cjsg` próprio e o `cjpg`, de primeiro grau

A busca no e-SAJ foi escrita uma vez, e não seis: uma correção nela vale para os seis tribunais. O TJSP tem diferenças, e o `cjsg` dele vale no lugar do herdado.

Gramática e vocabulário

Insper

Até aqui

Variáveis
Loops
Objetos

Condicionais
Dicionários

Listas
Funções

Até aqui

Variáveis
Loops
Objetos

Condicionais
Dicionários

Listas
Funções

É a gramática do Python: com ela se escreve qualquer frase. Junto vieram as ferramentas de quem programa: VS Code, terminal, Git, GitHub e o assistente.

Gramática não basta

Deblaterar

Gramática não basta

Deblaterar

falar com veemência contra alguém ou algo; bradar

Gramática não basta

Deblaterar

falar com veemência contra alguém ou algo; bradar

Estugar

apressar o passo; estimular, incitar

Bibliotecas aumentam o vocabulário

Vocês sabem a gramática do português, e mesmo assim não sabiam essas palavras. No Python é igual: a gramática vocês têm, e cada biblioteca traz palavras novas, prontas para importar e usar.

`jus.scraper`, `cjsg`, `raspe.camara` foram as primeiras. Quem sabe as palavras consegue dizer à IA, com precisão, o que quer.

Cada área tem o seu vocabulário

Área	Palavras que quem é de fora não usa
Direito	preclusão, litispendência, agravo interno
Medicina	anamnese, etiologia, prognóstico
Computação	compilador, cache, concorrência

Ninguém aprende todos. Cada um aprende o da área em que trabalha, e as bibliotecas também se dividem por área.

Os dois vocabulários do resto do curso

Bibliotecas	Para quê
para usar IA: LangChain	pedir a um modelo de linguagem, de dentro do programa, que leia, classifique e extraia
para bases de dados: pandas e as de gráfico	ler, filtrar, juntar e resumir tabelas, e desenhar o resultado

Em CDAD 2: conceitos de estatística e as bibliotecas de estatística.

Bibliotecas para usar IA

Insper

Um produto com cliente

O produto da fase 2 vai ter quem use e quem pague por ele.

Com o que o grupo precisa se preocupar aqui que num exercício de aula não precisaria?

Um produto com cliente

RESPOSTA

preocupação	o que o cliente pergunta
custo	quanto vou pagar para rodar a base?
velocidade	em quanto tempo sai o resultado?
experiência de quem usa	quem vai usar consegue rodar e entender?
custo-efetividade	o resultado vale o que custa?

Tudo isso começa em duas decisões: usar um modelo de linguagem ou não e, se sim, qual.

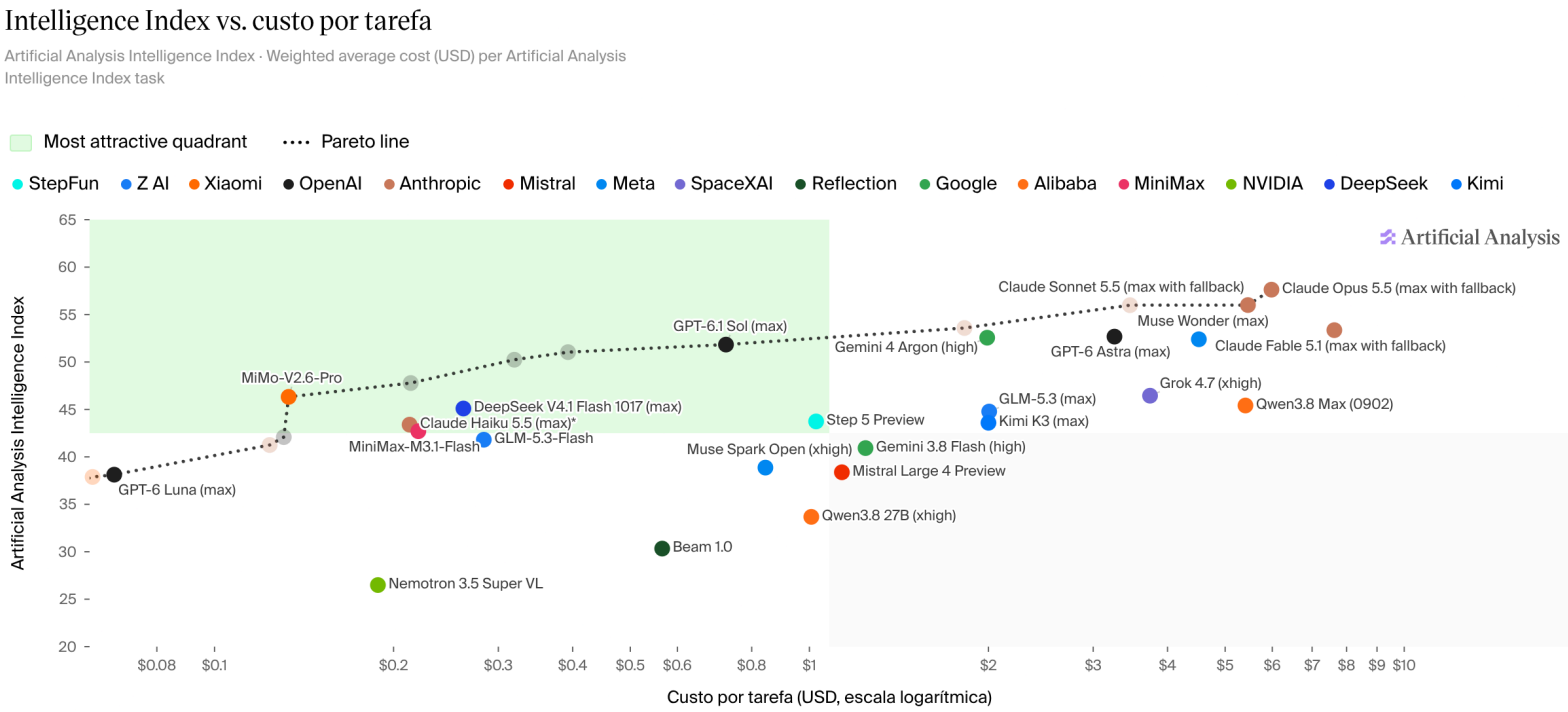
Onde comparar modelos

O Artificial Analysis (artificialanalysis.ai) testa os modelos por conta própria e publica três medidas:

medida	como sai
inteligência	um índice de 0 a 100, a partir de baterias de testes
velocidade	tokens por segundo na resposta
preço	dólares por milhão de tokens, ou por tarefa do teste

O índice mede testes gerais, e não a tarefa do grupo. Só testar numa amostra diz se o modelo serve para o produto.

Inteligência contra preço



Artificial Analysis, 21/10/2026. A linha pontilhada é a fronteira.

Insper

A fronteira de Pareto

Um modelo está na fronteira quando nenhum outro mais barato é mais inteligente. Fora dela, sempre há um modelo melhor pelo mesmo preço, ou mais barato com a mesma inteligência.

No gráfico, a linha vai do GPT-6 Luna, perto de US$ 0,07 por tarefa, ao Claude Opus 5.5, perto de US$ 6, passando pelo MiMo-V2.6-Pro, pelo GPT-6.1 Sol e pelo Claude Sonnet 5.5. O quadrado verde é o canto de mais inteligência por menos dinheiro.

É a fronteira de 21/10/2026. Sai modelo novo toda semana: consulte o site no dia da escolha.

Onde rodar

onde	o que dá de graça
Groq, plano gratuito	sem cartão; 1.000 pedidos por dia; é o da demonstração
Google Cloud	US$ 300 por 90 dias, com cartão
AWS	US$ 100, mais até US$ 100 por tarefas, por 12 meses
Azure for Students	US$ 100 por 12 meses, sem cartão

Nas nuvens, os créditos pagam modelos de vários fornecedores. Se o cliente exigir que os dados fiquem no Brasil, pergunte ao provedor, modelo a modelo: a região de São Paulo existe, mas nem todo modelo roda nela.

Cem sentenças para classificar

Em quantas o pedido foi acolhido?

Uma advogada de consumidor raspou cem sentenças sobre negativa de cobertura de plano de saúde. Ela quer saber em quantas o juiz acolheu o pedido do paciente.

A resposta está no texto de cada sentença, e cada juiz escreve do seu jeito.

Procurar a palavra "procedente"

```
sentencas = [
    "Julgo procedente o pedido.",
    "Julgo improcedente o pedido.",
    "Julgo parcialmente procedente o pedido.",
]
for texto in sentencas:
    print("procedente" in texto)
```

O `in` que vocês usaram com listas também procura um trecho dentro de um texto. O que aparece?

O trecho dentro da palavra

RESPOSTA

```
True
True
True
```

O `in` acha "procedente" em qualquer lugar do texto: dentro de "improcedente" e de "parcialmente procedente" também.

Testar "improcedente" primeiro

```
sentencas = [
    "Julgo improcedente o pedido.",
    "Rejeito o pedido da autora.",
    "O pedido não merece acolhida.",
]
for texto in sentencas:
    if "improcedente" in texto:
        print("improcedente")
    elif "procedente" in texto:
        print("procedente")
    else:
        print("?")
```

O que aparece?

Três sentenças improcedentes, uma reconhecida

RESPOSTA

```
improcedente
?
?
```

As três negam o pedido. Duas dizem isso sem a palavra "improcedente", e cada forma nova de escrever pediria mais um `if`. "Parcialmente procedente" cairia em "procedente". A lista de regras não acaba.

O modelo dentro do programa

Insper

Cem sentenças no chat

Uma sentença, vocês colam no chat e perguntam se o pedido foi acolhido.

Como fariam com as cem? E com dez mil?

O programa pergunta ao modelo

RESPOSTA

O programa manda cada sentença ao modelo sozinho, num `for`. Cada envio é um pedido pela internet a um computador do provedor, que devolve uma resposta:



O pedido leva o prompt e a chave; a resposta traz o texto. É isso que o chat faz por trás da tela, e é o que se chama de API.

Por que o provedor pede uma chave

Cada resposta custa ao provedor tempo de computador. A chave diz quem está pedindo: a conta de quem paga, ou de quem usa a cota gratuita.

Sem a chave, ou com uma chave que não vale, o provedor recusa o pedido. Com a chave, ele conta quantos pedidos a conta fez e recusa os que passam do limite.

Uma chave só para a turma inteira

Eu crio uma chave e passo para os 55. No mesmo minuto, cada um manda três sentenças ao modelo.

A conta gratuita aceita cerca de 30 pedidos por minuto. O que acontece?

O limite é da conta

RESPOSTA

São 165 pedidos no mesmo minuto. Do trigésimo primeiro em diante, o provedor recusa: a resposta volta com um erro de limite.

E quem tem a chave usa a conta em nome do dono dela. Por isso cada um cria a sua.

Criar a conta e a chave na Groq

#	O que fazer
1	Abrir `console.groq.com` e entrar com a conta do Google, do GitHub ou um e-mail
2	No menu, **API Keys**, e depois **Create API Key**; dar um nome, como `cdad1`
3	Copiar a chave agora: ela começa com `gsk_`

A chave é sua. Não mandem a ninguém, nem para mim.

Onde guardar a chave

O jeito mais rápido é colar a chave no próprio programa:

```
chave = "gsk_..."
```

O programa funciona. O que pode dar errado?

A chave escrita no código

Animação no navegador: `animacao-api-llm.html`, cenário `codigo`.

A chave vai junto com o arquivo

RESPOSTA

O arquivo vai para o repositório, e a chave vai junto. Apagar a linha e enviar de novo não resolve: a versão anterior continua no histórico.

Chave que vazou se apaga no painel da Groq, em **API Keys**, e se troca por outra.

A chave no arquivo .env

#	O que fazer, na pasta `cdad1`
1	No terminal do VS Code: `uv add python-dotenv langchain-groq`
2	Criar o arquivo `.env`, com uma linha só: `GROQ_API_KEY=` e a sua chave colada depois do =
3	Abrir o arquivo `.gitignore` e acrescentar uma linha com `.env`

O `.gitignore` lista o que o Git não registra: o `.env` fica no computador e nunca vai para o GitHub.

Conferir a chave

```python
import os
from dotenv import load_dotenv

load_dotenv()
chave = os.environ["GROQ_API_KEY"]
print(chave[:4])
```

Rodem num notebook da pasta `cdad1`. O que aparece? E se o `.env` não existisse?

os.environ lê o ambiente

RESPOSTA

```
gsk_
```

O `load_dotenv()` leva o que está no `.env` para o ambiente, e `os.environ` é um dicionário com ele. Sem o arquivo, a chave não existe:

```
KeyError: 'GROQ_API_KEY'
```

Só os quatro primeiros caracteres vão para a tela: a chave inteira nunca aparece num print.

A primeira chamada

O que é o LangChain

Uma biblioteca para conversar com modelos de linguagem. Cada provedor tem um pacote, e o resto do programa não muda:

Pacote	Classe	Modelos de
`langchain-groq`	`ChatGroq`	Groq
`langchain-openai`	`ChatOpenAI`	OpenAI
`langchain-anthropic`	`ChatAnthropic`	Anthropic

Trocar de modelo é trocar uma linha. E ele traz peças prontas para o que vem depois: prompts, histórico e ferramentas que o modelo pode usar.

Perguntar ao modelo pelo Python

```
import os
from dotenv import load_dotenv
from langchain_groq import ChatGroq
load_dotenv()
modelo = ChatGroq(model="openai/gpt-oss-120b",
                  api_key=os.environ["GROQ_API_KEY"])
pergunta = "Rejeito o pedido. Procedente ou improcedente?"
resposta = modelo.invoke(pergunta)
print(resposta.content)
```

Rodem. O que aparece?

Objeto, método e atributo

RESPOSTA

Aparece o texto que o modelo escreveu, e cada um viu o seu.

No código	O que é
`ChatGroq(...)`	cria o objeto do modelo
`modelo.invoke(...)`	o método que manda o pedido
`resposta`	um objeto, com o texto e os tokens
`resposta.content`	o atributo com o texto

O pedido e a resposta, passo a passo

Animação no navegador: `animacao-api-llm.html`, cenário `ambiente`.

Agora, praticar

#	Com o programa da primeira chamada
1	Troquem a pergunta por uma das sentenças da lista e peçam uma palavra só: procedente, improcedente ou parcial
2	Repitam com as outras duas sentenças
3	Comparem com o vizinho: as respostas batem?

Na próxima aula: retomamos a chamada para entender a API e como o modelo funciona por dentro, até as ferramentas que ele pode usar.